O GUARDIÃO DOS DADOS



MANUAL TÉCNICO

Quiz web sobre segurança da informação e LGPD: funcionamento, tecnologias, funções, páginas, variáveis e onde alterar cada coisa.

Autor: Bruno Fioravanti

Projeto: Atividade Extensionista II — Tecnologia Aplicada à Inclusão Digital

Gerado em: 13/11/2026

Este manual explica como o projeto está organizado, o que faz cada arquivo e cada função, e — principalmente — **onde mexer quando você quiser mudar alguma coisa**. Se estiver com pressa, vá direto à seção 11. Onde alterar cada coisa.

> Os números de linha citados valem para a versão atual do código. Se o arquivo mudar, procure pelo nome da função ou da variável (Ctrl+F), que continua o mesmo.

Sumário

1. Visão geral
2. Tecnologias usadas
3. Estrutura de pastas
4. Configurações e variáveis de ambiente
5. Banco de dados
6. Páginas e endereços (rotas)
7. Funções do Model (dados)
8. Funções do Controller (rotas, gráficos e PDF)
9. Templates (View/Página)
10. Estilo visual (View/Estilo/style.css)
11. Onde alterar cada coisa
12. Publicar, aplicar e documentar
13. Cuidados e armadilhas

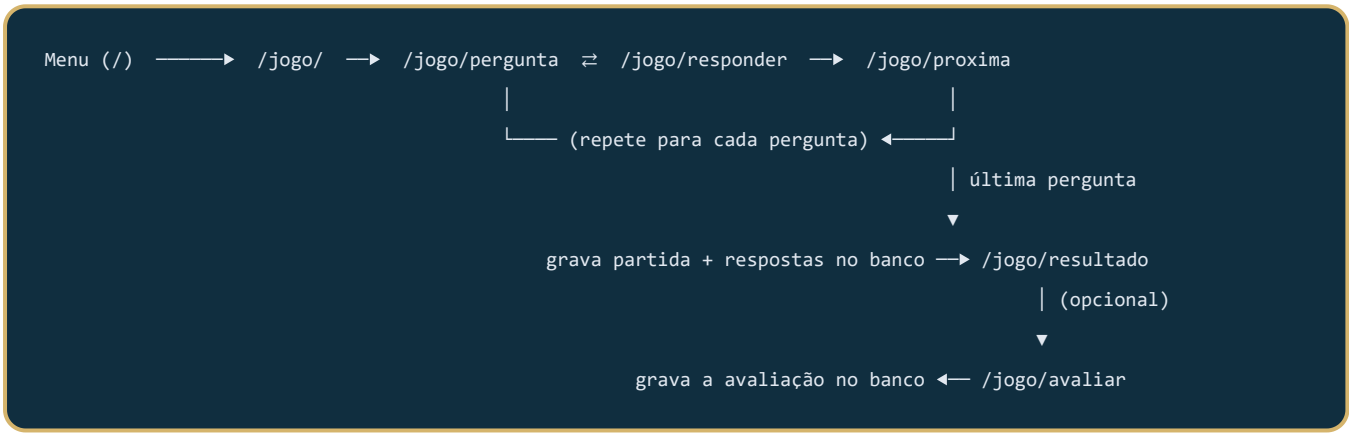
1. Visão geral

É um quiz web de cibersegurança e LGPD com tema medieval. A cada jornada são sorteadas 5 "provações" (perguntas de 5 alternativas) de um banco de perguntas; após cada resposta o Guardião explica o assunto, e no final o participante recebe um título e responde à **avaliação do quiz** (validação): três notas de 1 a 10 — dificuldade das perguntas, facilidade de uso e relevância das informações — e uma sugestão escrita opcional. Antes da avaliação, todos veem o quadro **"Por que isso importa?"**, que liga o conteúdo aos ODS 16 e 10 e à LGPD.

A participação é **anônima**: ninguém digita nome, e cada pessoa é registrada como "Jogador 1", "Jogador 2"... O menu explica, no quadro "Como seus dados são usados", exatamente o que é gravado.

Uma área administrativa protegida por PIN mostra estatísticas, o histórico de jornadas, as avaliações recebidas, gráficos e relatórios em PDF, e permite cadastrar perguntas.

Como uma jornada acontece, do clique ao banco:



- Em `/jogo/` são sorteadas as perguntas da jornada (5, sem repetir, em ordem aleatória).
- Enquanto o jogo acontece, o andamento (perguntas, posição atual, respostas dadas) fica **na sessão** do navegador (um cookie assinado). Nada vai para o banco no meio do caminho.
- Só quando a última pergunta é concluída a jornada é gravada (tabela `partidas`) junto com cada resposta (tabela `respostas`). Cada jornada concluída recebe o próximo número de jogador (tabela `jogadores`): a primeira é "Jogador 1", a segunda "Jogador 2", e assim por diante.
- Na tela de resultado, a avaliação enviada é gravada na tabela `avaliacoes`, ligada à jornada.
- É dessas tabelas que saem todas as estatísticas, gráficos e PDFs da área administrativa.

O que o participante vê, em ordem: menu (com o aviso de privacidade) → 5 perguntas, cada uma seguida da explicação do Guardião → resultado (título conquistado, placar, resumo e o aviso "⬇️ Sua jornada não termina aqui!") → quadro "Por que isso importa?" → avaliação do quiz → "Nova Jornada" ou "Voltar à Cidadela".

O código segue o padrão **MVC**:

Camada	Pasta	Responsabilidade
Model	`Model/`	Configuração, conexão com o banco e todas as consultas SQL
View	`View/`	O que aparece na tela: HTML (templates Jinja) e CSS
Controller	`Controller/`	Recebe cada requisição, chama o Model e escolhe qual View mostrar

2. Tecnologias usadas

Tecnologia	Onde aparece	O que constrói no projeto
Python 3.12	todo o back-end	Linguagem do servidor.
Flask (`requirements.txt`)	`app.py`, `Controller/`	Framework web: recebe as requisições, define as rotas (URLs), cuida da sessão (cookie assinado) e renderiza os templates. Os *blueprints* (`menu`, `jogo`, `admin`) separam as rotas por assunto.
Jinja2 (vem com o Flask)	`View/Página/*.html`	Linguagem dos templates: `{{ variavel }}` imprime valores, `{% if %}` / `{% for %}` controlam o HTML, `{% extends %}` herda o layout de `base.html` e `{% macro %}` cria "componentes" reutilizáveis (Guardião, gráficos).
SQLite (vem com o Python)	`Model/banco.py`	Banco local em arquivo (`quiz.db`), usado quando você roda no seu computador. Criado sozinho na primeira execução.
PostgreSQL / Neon + **psycopg**	`Model/banco.py`	Banco de produção no Vercel. O Neon é um PostgreSQL na nuvem; o psycopg é a biblioteca Python que conversa com ele. Usado sempre que a variável `DATABASE_URL` existir.
ReportLab	`Controller/relatorio_pdf.py`	Gera os três relatórios em PDF da área admin (textos, tabelas, gráficos de pizza, de barras e de colunas) diretamente no servidor.
SVG (desenho vetorial no HTML)	`_guardiao.html`, `_graficos.html`, `graficos.py`	O personagem Guardião, o título medieval, o cenário do menu e os gráficos de pizza da tela são desenhados em SVG — não há imagens nem bibliotecas de gráficos JavaScript.
CSS	`View/Estilo/style.css`	Todo o visual: cores, tema pergaminho, botões, layout para celular, barras e colunas dos gráficos, escala de notas 1–10.
JavaScript (pequeno, embutido)	`base.html`, `jogo.html`, `admin_estatisticas.html`, `admin_avaliacoes.html`	Só detalhes: a janela de confirmação "⚜ O Guardião", os atalhos de teclado A–E/1–5 no jogo, a caixa "marcar todas" nas estatísticas e o filtro de sugestões. O jogo funciona sem JavaScript.
Google Fonts (Cinzel / Cinzel Decorative)	`base.html`	Fontes medievais dos títulos.
zoneinfo + tzdata	`Model/config.py`	Fuso horário: grava e mostra as datas no horário de Brasília mesmo com o servidor do Vercel em UTC. O `tzdata` fornece a tabela de fusos no Windows.
Vercel	`app.py` (variável `app`)	Hospedagem. O Vercel procura uma variável `app` no `app.py` e a executa como função *serverless*.
GitHub	repositório `O-Guardiao-dos-Dados`	Versionamento do código; o Vercel publica a partir dele.
Markdown + navegador (Edge/Chrome)	`docs/gerar_manual_pdf.py`	Só para gerar o PDF deste manual (não é usado pelo app).

3. Estrutura de pastas

```
O-Guardiao-dos-Dados/
├── app.py                    ← ponto de entrada: cria o app Flask (local e Vercel)
├── requirements.txt          ← bibliotecas Python necessárias
├── README.md                 ← como executar e publicar
├── MANUAL.md                 ← este manual
├── docs/
│   ├── Manual - O Guardião dos Dados.pdf ← este manual em PDF, no visual do projeto
│   ├── gerar_manual_pdf.py   ← gera o PDF a partir do MANUAL.md
│   ├── Apresentação - O Guardião dos Dados.pdf ← slides do projeto (16:9, ~10 min)
│   ├── Roteiro da apresentação.pdf             ← o que falar em cada slide
│   ├── gerar_apresentacao_pdf.py ← gera os dois PDFs acima (textos, dados e link ficam nele)
│   └── apresentacao/         ← capturas de tela do quiz usadas nos slides
├── quiz.db                   ← banco SQLite local (criado sozinho; não vai para o GitHub)
│
├── Model/
│   ├── config.py             ← TODAS as configurações (PIN, banco, fuso, letras…)
│   ├── banco.py              ← conexão SQLite/PostgreSQL e criação das tabelas
│   ├── repositorio.py        ← todas as consultas: perguntas, jogadores, partidas, avaliações
│   └── perguntas_iniciais.py ← o banco de perguntas padrão (25)
│
├── Controller/
│   ├── menu.py               ← tela inicial
│   ├── jogo.py               ← fluxo do quiz
│   ├── admin.py              ← área administrativa
│   ├── graficos.py           ← cálculo das fatias da pizza (SVG)
│   └── relatorio_pdf.py      ← geração dos PDFs
│
└── View/
    ├── Estilo/style.css      ← visual de todas as páginas (servido em /estilo/style.css)
    └── Página/               ← templates HTML
        ├── base.html                 ← layout comum a todas as páginas
        ├── menuprincipal.html        ← menu inicial
        ├── jogo.html                 ← tela de cada pergunta
        ├── resultado.html            ← fim da jornada
        ├── sem_perguntas.html        ← aviso quando não há perguntas
        ├── admin_login.html          ← pedido de PIN
        ├── admin.html                ← layout da área admin (abas)
        ├── admin_estatisticas.html   ← aba Estatísticas
        ├── admin_jornadas.html       ← aba Registro de Jornadas
        ├── admin_avaliacoes.html     ← aba Avaliações
        ├── admin_perguntas.html      ← aba Grimório de Perguntas (lista)
        ├── admin_form_pergunta.html  ← formulário de nova/editar pergunta
        ├── _guardiao.html            ← macros: personagem, cenário e título (SVG)
        └── _graficos.html            ← macros: pizza, barras, colunas de notas e percentual
```

Arquivos que começam com `_` não são páginas: são bibliotecas de macros importadas por outras páginas.

4. Configurações e variáveis de ambiente

Tudo fica em Model/config.py. Várias configurações podem vir de **variáveis de ambiente** (definidas no Vercel em *Settings → Environment Variables*). Quando a variável não existe, vale o valor padrão escrito no código.

Nome no código	Variável de ambiente	Padrão	Para que serve
`PIN_ADMIN`	`QUIZ_PIN_ADMIN`	`1234` (só para uso local)	PIN da área administrativa.
`URL_BANCO`	`DATABASE_URL` ou `POSTGRES_URL` (também com prefixo, ex.: `STORAGE_DATABASE_URL`)	—	Endereço do PostgreSQL (Neon). Se existir, o app usa PostgreSQL; se não, SQLite.
`CAMINHO_BANCO`	`QUIZ_BANCO`	`quiz.db` na raiz (ou `/tmp/quiz.db` no Vercel)	Arquivo SQLite. Útil para testes com um banco separado.
`FUSO`	`QUIZ_FUSO`	`America/Sao_Paulo`	Fuso das datas gravadas e exibidas.
`SECRET_KEY` (em app.py)	`QUIZ_SECRET_KEY`	aleatória a cada início	Assina o cookie da sessão. **No Vercel, defina-a**: sem ela, cada instância do servidor gera uma chave diferente e os jogadores perdem o progresso.
—	`VERCEL`	(o Vercel define sozinho)	Faz o SQLite usar `/tmp` (a única pasta gravável no Vercel) e liga os alertas de configuração do admin.
`BANCO_TEMPORARIO` / `SEM_CHAVE_SESSAO`	—	calculados	`True` quando o site está no Vercel sem PostgreSQL / sem `QUIZ_SECRET_KEY`. Mostram um alerta vermelho no topo da área administrativa.

Constantes que só existem no código (sem variável de ambiente):

Nome	Valor	Para que serve
`RAIZ`	pasta do projeto	Base para achar `View/` e `quiz.db`.
`LETRAS`	`['A','B','C','D','E']`	Letras exibidas nas alternativas.
`QTD_ALTERNATIVAS`	`5`	Quantas alternativas o formulário de pergunta exige.
`PERGUNTAS_POR_JORNADA`	`5`	Quantas perguntas são sorteadas a cada jornada (com menos cadastradas, usa todas). Também aparece na fala do menu.
`NOTA_MAXIMA`	`10`	Escala da avaliação do quiz (de 1 até este valor).
`TAMANHO_MAXIMO_COMENTARIO`	`1000`	Limite de caracteres da sugestão escrita.

Nome	Valor	Para que serve
`CRITERIOS_AVALIACAO`	3 itens	As perguntas da avaliação. Cada item tem `chave` (coluna do banco — não mude), `pergunta` (texto que o jogador vê), `titulo` (gráficos), `curto` (selos e colunas do PDF), `minimo` e `maximo` (o que significam as notas 1 e 10).

PIN em produção: o PIN escrito em `config.py` vai para o GitHub junto com o código. Para o site publicado, prefira definir `QUIZ_PIN_ADMIN` no Vercel — a variável de ambiente sempre tem prioridade sobre o valor do código.

5. Banco de dados

O esquema está em **Model/banco.py** (constantes `ESQUEMA` e `TABELAS_ADICIONAIS`). São cinco tabelas:

`perguntas` — o "grimório"

Coluna	Tipo	Conteúdo
`id`	inteiro, automático	Identificador.
`enunciado`	texto	A pergunta.
`alternativas`	texto	Lista JSON com as 5 alternativas, ex.: `["...", "...", ...]`.
`correta`	inteiro	Índice da alternativa certa (0 = A, 1 = B … 4 = E).
`comentario`	texto	"Sabedoria do Guardião", exibida após responder.

`partidas` — uma linha por jornada concluída

Coluna	Tipo	Conteúdo
`id`	inteiro, automático	Identificador.
`data`	texto	Data/hora com fuso, ex.: `2026-11-10T11:46:13-03:00`.
`total`	inteiro	Quantas perguntas foram respondidas.
`acertos`	inteiro	Quantas acertou.
`participante`	texto ou vazio	Rótulo anônimo do jogador, ex.: `Jogador 3` (vazio nas jornadas mais antigas).

`jogadores` — numeração anônima dos jogadores

Coluna	Tipo	Conteúdo
`id`	inteiro, automático	O número do jogador (o "3" de "Jogador 3"), um por jornada concluída. Gerado pelo banco, então duas jornadas terminando ao mesmo tempo nunca recebem o mesmo número.
`codigo`	texto	Código aleatório que só preenche a coluna (não identifica ninguém).

`avaliacoes` — avaliação do quiz feita ao final da jornada (validação)

Coluna	Tipo	Conteúdo
`id`	inteiro, automático	Identificador.
`partida_id`	inteiro, único	A jornada avaliada (no máximo uma avaliação por jornada; apagar a jornada apaga a avaliação).
`data`	texto	Data/hora do envio, com fuso.
`dificuldade`	inteiro (1–10)	Nível de dificuldade das perguntas (1 = muito fáceis, 10 = muito difíceis).

Coluna	Tipo	Conteúdo
`facilidade`	inteiro (1–10)	Facilidade de uso do quiz (1 = muito difícil, 10 = muito fácil).
`relevancia`	inteiro (1–10)	Relevância das informações recebidas (1 = nada, 10 = muito relevantes).
`comentario`	texto ou vazio	Sugestão escrita (opcional, até 1000 caracteres).

`respostas` — uma linha por pergunta respondida em cada jornada

Coluna	Tipo	Conteúdo
`id`	inteiro, automático	Identificador.
`partida_id`	inteiro	A jornada (apagar a jornada apaga suas respostas — *ON DELETE CASCADE*).
`pergunta_id`	inteiro	A pergunta respondida (sem vínculo obrigatório, para o histórico sobreviver à exclusão da pergunta).
`enunciado`	texto	Cópia do enunciado no momento da resposta.
`escolhida`	inteiro	Alternativa marcada (0–4).
`correta`	inteiro	Alternativa certa naquele momento (0–4).

Criação e atualização automáticas: ao iniciar, `banco.inicializar()` cria as tabelas se não existirem (e cadastra as perguntas padrão) e `_atualizar_esquema()` adiciona colunas e tabelas novas em bancos antigos. Você nunca precisa rodar SQL à mão — nem no `quiz.db`, nem no Neon.

Ver os dados: localmente, abra `quiz.db` com qualquer visualizador de SQLite (ex.: extensão *SQLite Viewer* do VS Code). No Neon, use o *SQL Editor* do painel do Neon (acessível pela aba *Storage* do Vercel).

6. Páginas e endereços (rotas)

Endereço	Método	Função	Template	O que faz
`/`	GET	`menu.inicio`	`menuprincipal.html`	Menu com o botão "Iniciar Provação".
`/jogo/`	GET	`jogo.iniciar`	— (redireciona)	Sorteia as perguntas da jornada e começa.
`/jogo/pergunta`	GET	`jogo.pergunta`	`jogo.html`	Mostra a pergunta atual (ou o feedback, se já respondida).
`/jogo/responder`	POST	`jogo.responder`	—	Registra a alternativa clicada na sessão.
`/jogo/proxima`	POST	`jogo.proxima`	—	Avança; na última pergunta grava a jornada no banco.
`/jogo/resultado`	GET	`jogo.resultado`	`resultado.html`	Placar, título conquistado, resumo das respostas e o formulário de avaliação (ou o agradecimento).
`/jogo/avaliar`	POST	`jogo.avaliar`	—	Recebe a avaliação da última jornada concluída e volta para o resultado.
`/admin/login`	GET/POST	`admin.login`	`admin_login.html`	Pede o PIN.
`/admin/sair`	GET	`admin.sair`	—	Sai da área admin.
`/admin/`	GET	`admin.inicio`	—	Redireciona para Estatísticas.
`/admin/estatisticas`	GET	`admin.estatisticas`	`admin_estatisticas.html`	Acerto por pergunta; com `?graficos=1` mostra as pizzas; `?p=<id>` filtra perguntas.
`/admin/estatisticas/pdf`	GET	`admin.estatisticas_pdf`	— (PDF)	Baixa o relatório de estatísticas.
`/admin/jornadas`	GET	`admin.jornadas`	`admin_jornadas.html`	Indicadores, média por jogador e lista de jornadas.
`/admin/jornadas/pdf`	GET	`admin.jornadas_pdf`	— (PDF)	Baixa o relatório de jornadas.
`/admin/avaliacoes`	GET	`admin.avaliacoes`	`admin_avaliacoes.html`	Indicadores, gráficos das notas e lista de avaliações/sugestões; `?comentarios=1` mostra só as com sugestão escrita.
`/admin/avaliacoes/pdf`	GET	`admin.avaliacoes_pdf`	— (PDF)	Baixa o relatório de avaliações.
`/admin/jornadas/limpar`	POST	`admin.limpar_jornadas`	—	Apaga todo o histórico (jornadas, respostas e avaliações).
`/admin/perguntas`	GET	`admin.perguntas`	`admin_perguntas.html`	Lista as perguntas.

Endereço	Método	Função	Template	O que faz
`/admin/perguntas/nova`	GET/POST	`admin.nova_pergunta`	`admin_form_pergunta.html`	Cadastra pergunta.
`/admin/perguntas/<id>/editar`	GET/POST	`admin.editar_pergunta`	`admin_form_pergunta.html`	Edita pergunta.
`/admin/perguntas/<id>/excluir`	POST	`admin.excluir_pergunta`	—	Exclui pergunta (o histórico dela é mantido).
`/admin/perguntas/adicionar-padrao`	POST	`admin.adicionar_perguntas_padrao`	—	Adiciona as perguntas padrão que ainda não estão no banco, sem apagar nem alterar nenhuma.
`/admin/perguntas/restaurar`	POST	`admin.restaurar_perguntas`	—	Troca **todas** as perguntas pelas padrão.
`/estilo/style.css`	GET	(Flask)	—	Arquivo de estilo.

Todas as rotas `/admin/...` exceto o login passam antes por `exigir_acesso`, que manda para o login quem não digitou o PIN.

7. Funções do Model (dados)

app.py

Função / item	O que faz
`criar_app()`	Cria o Flask apontando os templates para `View/Página` e o CSS para `View/Estilo` (servido em `/estilo`), define a chave da sessão, prepara o banco (`banco.inicializar`) e registra os três blueprints.
`app = criar_app()`	Instância usada pelo Vercel. **Não remova nem renomeie** — o Vercel procura exatamente `app`.
`if __name__ == '__main__'`	Só roda com `python app.py`: servidor local em `http://127.0.0.1:5000`, com modo debug (recarrega ao salvar arquivos).

Model/banco.py — conexão e esquema

Função / item	O que faz
`USA_POSTGRES`	`True` quando há `DATABASE_URL`; decide qual banco usar em todo o resto do código.
`ESQUEMA` / `COLUNA_ID`	O SQL de criação das tabelas; `{ID}` vira o tipo de chave automática de cada banco.
`class Conexao`	Deixa SQLite e PostgreSQL com a mesma "cara": `execute`, `executemany`, `commit`, `close`. Converte o marcador `?` (SQLite) para `%s` (PostgreSQL), então as consultas são escritas uma vez só.
`conectar()`	Abre uma conexão nova (PostgreSQL com resultados em dicionário; SQLite com chaves estrangeiras ligadas).
`obter()`	Conexão da requisição atual: abre na primeira consulta e reaproveita até o fim da requisição. É o que o repositório usa.
`fechar()`	Fecha a conexão no fim de cada requisição (registrado em `app.py`).
`inserir_perguntas_iniciais(conexao, perguntas)`	Grava perguntas no formato de `perguntas_iniciais.py` (por padrão, todas as de lá).
`_tabela_perguntas_existe(conexao)`	Diz se o banco já foi criado.
`_coluna_existe(conexao, tabela, coluna)`	Diz se uma coluna existe (usado nas atualizações de esquema).
`TABELAS_ADICIONAIS`	SQL das tabelas criadas depois da primeira versão (`jogadores` e `avaliacoes`). Para uma tabela nova, acrescente-a aqui.
`_atualizar_esquema(conexao)`	Roda sempre ao iniciar: adiciona `partidas.participante` se faltar e cria as `TABELAS_ADICIONAIS` que não existirem. Vale para bancos novos e para os que já existiam (quiz.db e Neon).
`inicializar()`	Roda uma vez ao iniciar: cria tudo se for um banco novo e aplica `_atualizar_esquema`. No PostgreSQL usa uma trava (*advisory lock*) para duas instâncias do Vercel não cadastrarem as perguntas em dobro.

Model/repositorio.py — todas as consultas

Datas e cálculo

Função	O que faz
`agora()`	Data/hora atual no fuso configurado. Usada ao gravar jornadas e no rodapé/nome dos PDFs.
`_ler_data(texto)`	Converte a data gravada para o fuso configurado. Datas antigas sem fuso são lidas como UTC (PostgreSQL) ou horário do computador (SQLite).
`percentual_acerto(acertos, total)`	Percentual inteiro, com 0,5 arredondando para cima; `None` se não houver respostas. **É a única fórmula de porcentagem do projeto** — tela, gráficos, resultado e PDF usam ela, por isso os números sempre batem.

Perguntas

Função	O que faz
`_montar_pergunta(linha)`	Transforma a linha do banco em dicionário (decodifica o JSON das alternativas).
`listar_perguntas()`	Todas as perguntas, na ordem de cadastro (o jogo sorteia a partir desta lista).
`obter_pergunta(id)`	Uma pergunta, ou `None`.
`contar_perguntas()`	Quantidade de perguntas (mostrada no menu).
`salvar_pergunta(...)`	Cria (sem `pergunta_id`) ou atualiza (com `pergunta_id`).
`excluir_pergunta(id)`	Apaga a pergunta; respostas antigas dela continuam nas estatísticas, marcadas como "(excluída)".
`restaurar_perguntas_iniciais()`	Apaga todas e grava as padrão.
`adicionar_perguntas_faltantes()`	Grava as perguntas padrão cujo enunciado ainda não está no banco; não apaga nada. Retorna quantas adicionou.

Jogadores

Função	O que faz
`novo_jogador()`	Devolve o próximo número de jogador (1, 2, 3...); chamado uma vez por jornada concluída.
`rotulo_jogador(numero)`	Texto gravado e exibido: `"Jogador N"`. Mude aqui para trocar o rótulo (ex.: "Viajante N").

Jornadas e estatísticas

Função	O que faz
`registrar_partida(respostas, participante)`	Grava a jornada (com o rótulo `Jogador N`) e cada resposta; devolve o id da jornada.
`_montar_resposta(linha)` / `_montar_partida(linha, respostas)`	Montam os dicionários usados pelas telas (`acertou`, `erros`, `taxa`, data no fuso...).

Função	O que faz
`obter_partida(id)`	Uma jornada com suas respostas (tela de resultado).
`listar_partidas()`	Todas as jornadas, mais recentes primeiro, cada uma com `numero` (#1 = a mais antiga). Faz só 2 consultas no total.
`medias_por_participante(partidas)`	Agrupa as jornadas pelo rótulo (hoje cada jornada tem um número próprio, então cada barra é uma jornada) e calcula a média de aproveitamento, acertos por jornada e número de jornadas. Jornadas antigas sem rótulo viram o grupo "Sem identificação", sempre por último.
`resumo_partidas()`	Totais gerais: jornadas, respostas, acertos, erros e taxa.
`estatisticas_por_pergunta()`	Acertos/erros/taxa de cada pergunta, somados em SQL a partir de `respostas`. Inclui perguntas sem respostas e perguntas já excluídas que têm histórico.
`limpar_partidas()`	Apaga todas as jornadas (e, em cascata, as respostas e avaliações) e recomeça a numeração do "Jogador 1".

Avaliações do quiz

Função	O que faz
`registrar_avaliacao(partida_id, notas, comentario)`	Grava a avaliação de uma jornada; se ela já foi avaliada, ignora (vale a primeira).
`obter_avaliacao(partida_id)`	A avaliação de uma jornada, ou `None` (a tela de resultado usa para mostrar o formulário ou o agradecimento).
`listar_avaliacoes()`	Todas as avaliações, mais recentes primeiro, com o jogador da jornada.
`resumo_avaliacoes(avaliacoes)`	Para cada critério: média (uma casa decimal) e distribuição (quantas avaliações deram cada nota de 1 a 10).

Model/perguntas_iniciais.py

Lista `PERGUNTAS_INICIAIS` com as 25 perguntas padrão, em três grupos (comentários no arquivo): LGPD — agentes, fiscalização e direitos; ODS 16 e proteção de dados; golpes e boas práticas do dia a dia. Cada item tem `enunciado`, `alternativas` (5 textos), `correta` (índice 0–4) e `comentario`. É usada na criação do banco e nos botões "Adicionar provações padrão que faltam" e "Restaurar provações padrão".

As alternativas erradas foram escritas com tamanho parecido com o da correta, para que o participante não acerte só escolhendo o texto mais longo. Mantenha esse cuidado ao escrever perguntas novas.

8. Funções do Controller (rotas, gráficos e PDF)

Controller/menu.py

Função	O que faz
`inicio()`	Renderiza o menu com o total de perguntas do banco e quantas são sorteadas por jornada.

Controller/jogo.py

A sessão guarda `session['jogo'] = {'perguntas': [ids sorteados], 'indice': posição, 'respostas': [...]}`, além de `session['ultima_partida']` (id da jornada, para a tela de resultado).

Função	O que faz
`_pergunta_atual(jogo)`	Pergunta da posição atual; pula as que o admin excluiu durante a partida.
`_finalizar(jogo)`	Tira o jogo da sessão, grava a jornada com o próximo número ("Jogador N") e manda para o resultado.
`iniciar()`	Sorteia `PERGUNTAS_POR_JORNADA` perguntas (sem repetir, em ordem aleatória) e começa.
`pergunta()`	Mostra a pergunta atual e, se já respondida, o feedback e o botão de avançar.
`responder()`	Valida e guarda a alternativa escolhida (impede responder duas vezes).
`proxima()`	Avança para a próxima pergunta ou finaliza.
`resultado()`	Mostra a tela final da última jornada concluída.
`_ultima_partida()`	A jornada guardada em `session['ultima_partida']`, ou `None`.
`_tela_resultado(partida, erro, preenchido)`	Escolhe a **mensagem** final ("Perfeito!", "Muito bem!"...) e monta a tela com o formulário de avaliação (ou o agradecimento).
`avaliar()`	Valida as 3 notas (1 a `NOTA_MAXIMA`) e a sugestão; se faltar nota, devolve a tela com o erro sem perder o que foi digitado.

Controller/admin.py

Função	O que faz
`add_app_template_global(...)` (linhas 17–18)	Deixa `fatias_pizza` e `percentual_acerto` disponíveis dentro dos templates e macros.
`exigir_acesso()`	Bloqueia a área admin para quem não fez login.
`login()` / `sair()` / `inicio()`	Entrada (compara o PIN de forma segura), saída e redirecionamento inicial.
`_estatisticas_selecionadas()`	Lê as perguntas marcadas (`?p=`) e devolve todas, as marcadas e as que entram nos gráficos/PDF (sem marcação = todas).

Função	O que faz
`estatisticas()` / `estatisticas_pdf()`	Tela e PDF de acerto por pergunta.
`jornadas()` / `jornadas_pdf()`	Tela e PDF do registro de jornadas (média geral, melhor resultado, médias por jogador).
`limpar_jornadas()`	Botão "Limpar histórico".
`_dados_avaliacoes()`	Junta as avaliações, o resumo por critério, a taxa de jornadas avaliadas e quantas têm sugestão (usado na tela e no PDF).
`avaliacoes()` / `avaliacoes_pdf()`	Tela e PDF da aba Avaliações.
`_baixar_pdf(conteudo, nome)`	Envia o PDF como download chamado `guardiao-<nome>-<data>.pdf`.
`perguntas()`	Lista do grimório.
`_ler_formulario()`	Lê e valida o formulário de pergunta; devolve os dados e a lista de erros.
`_formulario(...)`	Renderiza o formulário (vazio, preenchido ou com erros).
`nova_pergunta()` / `editar_pergunta(id)` / `excluir_pergunta(id)`	Cadastro de perguntas.
`adicionar_perguntas_padrao()` / `restaurar_perguntas()`	Botões "Adicionar provações padrão que faltam" (não apaga nada) e "Restaurar provações padrão" (substitui todas).

Controller/graficos.py

Função	O que faz
`fatias_pizza(acertos, erros)`	Calcula o desenho SVG de cada fatia (verde = acertos, vermelho = erros), começando no topo, no sentido horário. Devolve lista vazia quando não há respostas. Usada pela macro `pizza` em `_graficos.html`.
`estilo_percentual(propriedade, valor)`	Gera o atributo `style` das barras (ex.: `style="width: 80.0%"`) — gráficos, barra de taxa e barra de progresso do jogo. Aceita só `width`, `height` e `left`, e só números de 0 a 100. Registrada em `app.py` para todos os templates.

Controller/relatorio_pdf.py

Constantes no topo: `NOME_APP` (título dos PDFs), `LARGURA_UTIL` (largura útil da página A4), as **cores** (`AZUL`, `VERDE`, `VERMELHO`, `OURO`..., iguais às do CSS) e os **estilos de texto** (`ESTILO_TITULO`, `ESTILO_SECAO`...: fonte, tamanho, cor, espaçamento).

Função	O que faz
`relatorio_estatisticas(resumo, estatisticas, filtrado)`	**PDF de estatísticas**: visão geral (pizza total), tabela de acerto por pergunta e uma pizza por pergunta.
`relatorio_jornadas(resumo, partidas, participantes)`	**PDF de jornadas**: visão geral, gráfico de barras e tabela por participante, tabela de todas as jornadas.

Função	O que faz
`relatorio_avaliacoes(...)`	**PDF de avaliações**: visão geral, média de cada pergunta, gráfico de distribuição das notas de cada critério e tabela com as notas e sugestões.
`_grafico_notas(criterio)`	Gráfico de colunas (quantas pessoas deram cada nota).
`_media_nota(media)`	Média com vírgula (`7,4`) ou `–`.
`_gerar(titulo, elementos)`	Monta o documento A4 com cabeçalho (nome do app, título, data de geração) e rodapé.
`_rodape(canvas, documento)`	Escreve o rodapé "O Guardião dos Dados · título" e o número da página.
`_resumo_geral(resumo, linhas_extras)`	Bloco "Visão geral" (pizza + números).
`_pizza(acertos, erros, diametro)`	Desenha uma pizza (ou um círculo cinza "Sem respostas").
`_legenda(acertos, erros)`	Tabelinha colorida Acertos/Erros/Total.
`_bloco_grafico(...)`	Coloca a pizza à esquerda e um conteúdo à direita.
`_barras_participantes(participantes, media_geral)`	Gráfico de barras horizontais por participante com a linha tracejada da média geral (divide em blocos de 18 por página).
`_tabela_dados(cabecalho, linhas, larguras, colunas_numericas)`	Tabela padrão dos relatórios (cabeçalho bege, números alinhados à direita).
`_rotulo_participante(p)`	Rótulo do jogador ou "Sem identificação".
`_percentual(taxa)`	`"80%"` ou `"–"`.
`_seguro` / `_texto` / `_encurtar`	Ajustam textos para a fonte do PDF (troca caracteres não suportados, como emojis), escapam símbolos e cortam com "…" o que não cabe.

9. Templates (View/Página)

Layout base — base.html

Todas as páginas "herdam" deste arquivo com `{% extends 'base.html' %}` e preenchem os **blocos**:

Bloco	Para que serve
`titulo`	Texto da aba do navegador.
`classe_body`	Classe do `<body>` (`tela-interna` por padrão; o menu usa `tela-menu`).
`corpo`	Página inteira — o menu substitui tudo; as outras páginas mantêm o topo padrão.
`topo_titulo` / `topo_direita`	Texto no centro e à direita da barra superior.
`abaixo_topo`	Logo abaixo do topo (a barra de progresso do jogo).
`conteudo`	O conteúdo principal da página.
`scripts`	JavaScript da página, no fim do `<body>`.

Também carrega as fontes do Google e o CSS (`style.css?v=12` — veja Cuidados) e contém a **janela de confirmação "⚜️ O Guardião"** (`<dialog id="dialogo-confirmacao">` e o script logo abaixo). Todo formulário com o atributo `data-confirmar="mensagem"` abre essa janela antes de ser enviado; ela substitui o `confirm()` do navegador, cujo título ("Code" no navegador do VS Code, ou o endereço do site) não pode ser alterado.

Páginas do jogo

Template	Elementos principais
menuprincipal.html	Botão de engrenagem (admin) no canto, cenário gigante do Guardião, insígnia "Ordem da Cibersegurança & LGPD", título medieval, **fala de boas-vindas** (com o número de provações vindo de `PERGUNTAS_POR_JORNADA`), botão "Iniciar Provação", o aviso de que a participação é anônima e o quadro expansível **" 🔒 Como seus dados são usados"** (aviso de privacidade).
jogo.html	Barra de progresso, Guardião com expressão conforme a resposta (`indagando` / `orgulhoso` / `alerta`), balão com o enunciado, botões das alternativas, feedback ("Escudo Intacto!" / "Cuidado, viajante!"), comentário "Sabedoria do Guardião" e botão de avançar. Script de atalhos A–E / 1–5.
resultado.html	De cima para baixo: número do jogador e **título conquistado** (patente) conforme o percentual; placar e mensagem; aviso **" ⬇️ Sua jornada não termina aqui!"** (link que rola até o quadro seguinte; depois da avaliação enviada, deixa de convidar a avaliar); lista de acertos/erros; quadro **" 🌎 Por que isso importa?"** (ODS 16, ODS 10 e LGPD); cartão **" 📝 Avalie sua jornada"** (3 escalas de 1 a 10 e campo de sugestão; depois de enviado, mostra "Avaliação registrada"); botões "Voltar à Cidadela" / "Nova Jornada".
sem_perguntas.html	Aviso "Grimório Vazio" quando não há perguntas cadastradas.

Páginas da administração

Template	Elementos principais
admin_login.html	Campo de PIN.
admin.html	Layout da área admin: botão "Sair" e as quatro **abas**. A aba ativa é descoberta pelo nome da rota.
admin_estatisticas.html	4 indicadores (respostas, acertos, erros, taxa), tabela com caixa de seleção por pergunta e barra de taxa, botões "Gerar gráficos de pizza" e "Exportar PDF", grade de pizzas. Script "marcar todas".
admin_jornadas.html	3 indicadores (jornadas, média, melhor), cartão "Média por Participante" (barras), botões "Exportar PDF" e "Limpar histórico", lista expansível de jornadas (jogador, data, acertos/erros, %, respostas dadas).
admin_avaliacoes.html	3 indicadores (avaliações recebidas, % de jornadas avaliadas, quantas têm sugestão), cartão "Notas da Validação" com um gráfico por critério e o botão "Exportar PDF", e a lista "Avaliações e Sugestões" com o filtro "só as que têm sugestão escrita".
admin_perguntas.html	Total de perguntas, lista com "Editar"/"Excluir", botões "Adicionar provações padrão que faltam", "Restaurar provações padrão" e "+ Nova provação", aviso de quantas foram adicionadas.
admin_form_pergunta.html	Formulário: enunciado, 5 alternativas com marcação da correta, comentário.

Macros (componentes reutilizáveis)

Arquivo	Macro	O que desenha
_guardiao.html	`guardiao_svg(humor, classe)`	O medalhão do Guardião. `humor` muda olhos, boca e aura: `indagando`, `orgulhoso`, `alerta` (qualquer outro valor, como `saudacao`, usa a expressão neutra).
	`guardiao_fundo_menu()`	O Guardião gigante, torres e halo atrás do menu.
	`titulo_medieval()`	O brasão com "O GUARDIÃO / DOS DADOS".
_graficos.html	`percentual(taxa)`	Escreve `80%` ou `–`.
	`pizza(acertos, erros, titulo, subtitulo)`	Um gráfico de pizza com legenda.
	`barras_participantes(participantes, media_geral)`	O gráfico de barras da média por participante com a linha tracejada da média geral.
	`media_nota(media)`	Média com vírgula (`7,4`) ou `–`.
	`grafico_notas(criterio, nota_maxima)`	Gráfico de colunas da aba Avaliações: quantas pessoas deram cada nota, com a média no topo.

10. Estilo visual (View/Estilo/style.css)

Cores e fontes ficam em variáveis no início do arquivo (bloco `:root`). Trocar um valor ali muda o site inteiro:

Variável	Cor atual	Onde aparece
`--primaria` / `--primaria-escura`	azul aço / azul noite	Botões principais, títulos, barras de participantes.
`--ouro`, `--ouro-claro`, `--ouro-brilho`	dourados	Bordas, destaques, linha da média geral.
`--fundo-card-interno`, `--superficie`	tons de pergaminho	Fundo dos cartões e trilhos das barras.
`--texto`, `--texto-suave`	grafite / cinza	Textos.
`--borda`, `--borda-ouro`	bege / dourado claro	Linhas e contornos.
`--sucesso` / `--sucesso-claro`	verde	Acertos.
`--erro` / `--erro-claro`	vermelho	Erros.
`--raio`, `--sombra`	—	Arredondamento e sombra dos cartões.
`--fonte-epica`	Cinzel	Fonte dos títulos.

O fundo azul da página está no seletor `body` (gradiente logo abaixo de `:root`).

Seções do arquivo (procure pelos comentários `/* ---------- ... ---------- */`):

Seção	O que estiliza
Emblema de Título Medieval	O brasão do menu.
Insígnias e Personagem	Tamanhos e animações do Guardião (`.guardiao-destaque`, `.guardiao-dialogo`...).
Botões	`.botao-primario`, `.botao-secundario`, `.botao-perigo`, `.botao-grande`, `.botao-pequeno`.
Cartões	`.cartao` (o "pergaminho" branco).
Menu principal	Cenário, cartão do menu, botão da engrenagem.
Telas internas	Barra superior (`.topo`) e barra de progresso.
Jogo & Diálogo	Balão de fala, alternativas (`.correta`, `.incorreta`, `.esmaecida`), feedback.
Resultado	Placar e resumo.
Administração	Abas, indicadores, tabelas, lista de jornadas e perguntas.
Formulários	Campos de texto e alternativas.
`@media (max-width: 600px)` (1º)	Ajustes de celular das telas do jogo.
Quadro "Por que isso importa?", aviso de privacidade e janela de confirmação	`.aviso-rolar` (aviso com seta animada; a animação é desligada para quem pede menos movimento no sistema), `.grade-importancia`, `.selo-ods` (cores oficiais dos ODS 16 e 10), `.aviso-privacidade`, `.dialogo`.

Seção	O que estiliza
Avaliação do quiz (tela de resultado) e aba Avaliações	Botões da escala 1–10 (`.escala`, `.nota`), gráficos de colunas (`.grafico-notas`, `.barra-nota`), lista de sugestões (`.item-avaliacao`) e o ajuste de celular deles.
Estatísticas, gráficos de pizza e registro de jornadas	Pizzas, barras de participantes, aviso verde (`.aviso`) e, no fim, os ajustes de celular dessas telas (`@media 760px` e `600px`).

11. Onde alterar cada coisa

Acesso e configuração

Quero mudar...	Onde
PIN do admin	Vercel (produção): variável `QUIZ_PIN_ADMIN` em *Settings* → *Environment Variables* + *Redeploy*. Local: o padrão `1234` em **Model/config.py linha 10** — ou defina `QUIZ_PIN_ADMIN` no terminal antes de `python app.py` (PowerShell: `$env:QUIZ_PIN_ADMIN = '...'`). Não grave o PIN real no código: o repositório é público.
Fuso horário	Variável `QUIZ_FUSO` ou **config.py linha 44** (ex.: `America/Manaus`).
Quantas perguntas por jornada	**config.py** `PERGUNTAS_POR_JORNADA` (a fala do menu acompanha).
Rótulo "Jogador N"	`rotulo_jogador` em repositorio.py.
Porta do servidor local	**app.py, última linha** (`port=5000`).
Banco usado	Sem `DATABASE_URL` → SQLite `quiz.db`; com `DATABASE_URL` → PostgreSQL.

Textos e conteúdo

Quero mudar...	Onde
Perguntas padrão (as que vêm de fábrica)	**Model/perguntas_iniciais.py**. Entram sozinhas só em bancos novos. Num banco que já existe (o `quiz.db` ou o Neon), depois de acrescentar perguntas ao arquivo, use **Grimório → Adicionar provações padrão que faltam**.
Perguntas do jogo atual	Área admin → aba **Grimório de Perguntas** (não é preciso mexer no código).
Fala de boas-vindas do menu	**menuprincipal.html**. O número de provações vem de `PERGUNTAS_POR_JORNADA`.
"Iniciar Provação" e o aviso de anonimato	**menuprincipal.html**.
Insígnia "Ordem da Cibersegurança & LGPD"	**menuprincipal.html linha 17**.
Frases do jogo ("O Guardião indaga", "Escudo Intacto!"...)	**jogo.html**.
Títulos conquistados (patentes) e as faixas de %	**resultado.html linhas 9–21** (100% = Mestre, ≥70 = Paladino, ≥40 = Escudeiro, resto = Aprendiz).
Mensagem final ("Muito bem!", "Continue praticando!")	**Controller/jogo.py, função** `_tela_resultado`. Use as mesmas faixas das patentes.
Texto das abas do admin	**admin.html**.

Quero mudar...	Onde
Perguntas da avaliação (texto, títulos dos gráficos, significado das notas 1 e 10)	`CRITERIOS_AVALIACAO` em config.py. Não mude a `chave`.
Escala da avaliação (1 a 10)	`NOTA_MAXIMA`.
Aviso "Sua jornada não termina aqui!" (indica que há informações e a avaliação mais abaixo)	resultado.html, link `aviso-rolar`.
Quadro **"Por que isso importa?"** (ODS 16, ODS 10, LGPD e a frase final)	resultado.html, seção `cartao-importancia`.
Aviso de privacidade ("Como seus dados são usados" e o responsável pelo projeto)	menuprincipal.html, bloco `aviso-privacidade`. Se mudar o que o app grava, atualize este texto.
Título e botões da janela de confirmação	base.html, `<dialog id="dialogo-confirmacao">`. As mensagens ficam no `data-confirmar` de cada formulário (admin_jornadas.html e admin_perguntas.html).
Textos "Avalie sua jornada", "Avaliação registrada" e do campo de sugestão	resultado.html, cartão `#avaliacao`.
Texto "Sem identificação" (jornadas antigas sem rótulo)	Tela: _graficos.html e admin_jornadas.html; PDF: `_rotulo_participante` e `relatorio_jornadas`.

Nome do aplicativo

"O Guardião dos Dados" aparece em vários lugares; para renomear, troque em todos:

- bloco `titulo` de cada template (`base.html`, `admin*.html`, `jogo.html`, `resultado.html`, `sem_perguntas.html`);
- o brasão em _guardiao.html, macro `titulo_medieval` (textos "O GUARDIÃO" e "DOS DADOS" do SVG e o `<h1>` escondido usado por leitores de tela);
- `NOME_APP` em relatorio_pdf.py (título e rodapé dos PDFs);
- o prefixo do arquivo baixado `guardiao-` em `_baixar_pdf`;
- títulos do `README.md` e deste manual.

Dica: no VS Code, **Ctrl+Shift+F** procurando "Guardião dos Dados" lista todos.

Visual

Quero mudar...	Onde
Cores do site	Variáveis no topo do style.css (`:root`).
Cores dos PDFs	Constantes no topo do relatorio_pdf.py (mantenha iguais às do CSS).
Cor das colunas dos gráficos de avaliação	`.barra-nota` no style.css (tela) e `_grafico_notas` em relatorio_pdf.py (PDF).
Cor das pizzas na tela	`.fatia-acertos` / `.fatia-erros` no style.css (seção de estatísticas).

Quero mudar...	Onde
Cor das barras de participantes / linha da média	`.barra-preenchida`, `.barra-anonima .barra-preenchida`, `.barra-marca-media` no style.css.
Fontes dos títulos	Link do Google Fonts em **base.html** + `--fonte-epica` no CSS.
Desenho do Guardião	Macros em **_guardiao.html** (cada parte tem um comentário: pluma, elmo, barba, escudo...).
Layout no celular	Blocos `@media (max-width: 600px)` e `760px` do style.css.

Regras do jogo

Quero mudar...	Onde
Sorteio das perguntas	**`jogo.iniciar`** usa `random.sample` (sem repetição). Para voltar à ordem fixa de cadastro, troque o sorteio por `ids[:PERGUNTAS_POR_JORNADA]`.
Fórmula da porcentagem	**`percentual_acerto`** (vale para o site e os PDFs).
Média por jogador (agrupamento)	**`medias_por_participante`**.
Número de alternativas	Exige mexer em vários pontos juntos: `LETRAS` e `QTD_ALTERNATIVAS` em config.py, o atalho de teclado em **jogo.html** (`'ABCDE'` e `[1-5]`), as perguntas de `perguntas_iniciais.py` e as perguntas já cadastradas no banco.

Banco de dados

Quero...	Como
Adicionar uma coluna	1) inclua no `ESQUEMA` de **banco.py** (para bancos novos); 2) em **`_atualizar_esquema`** adicione um `if not _coluna_existe(...)` com o `ALTER TABLE` (para bancos já existentes, como o Neon); 3) ajuste as consultas em `repositorio.py`.
Zerar o banco local	Pare o servidor e apague o arquivo `quiz.db` — ele é recriado na próxima execução.
Zerar só o histórico	Área admin → Registro de Jornadas → **Limpar histórico**.

12. Publicar, aplicar e documentar

Publicar no Vercel (com o banco Neon)

1. No Vercel, conecte o projeto ao repositório **O-Guardiao-dos-Dados** (*Settings* → *Git*, ou crie um projeto novo a partir dele).
2. Em *Storage*, conecte um banco **Neon** — isso cria a variável `DATABASE_URL`.
3. Em *Settings* → *Environment Variables*, defina `QUIZ_SECRET_KEY` (gere com `python -c "import secrets; print(secrets.token_hex(32))"`) e `QUIZ_PIN_ADMIN` com o PIN real.
4. Faça o deploy. As tabelas são criadas sozinhas na primeira requisição.
5. Se o banco já existia de uma versão anterior, entre no admin → **Grimório de Perguntas** → **Adicionar provações padrão que faltam**, para receber as perguntas novas do código.

Antes de aplicar o quiz com o público

1. Teste uma jornada completa pelo celular, incluindo a avaliação.
2. Na área admin, use **Registro de Jornadas** → **Limpar histórico** para apagar as jornadas de teste. A numeração recomeça no "Jogador 1" e as estatísticas e avaliações ficam zeradas.
3. Divulgue o endereço do site (ex.: por QR Code). Não é preciso instalar nada nos aparelhos dos participantes.
4. Ao final da aplicação, exporte os três PDFs (Estatísticas, Registro de Jornadas e Avaliações): eles são a base dos gráficos do relatório final.

Atualizar o PDF deste manual

O arquivo `docs/Manual - O Guardião dos Dados.pdf` é gerado a partir deste `MANUAL.md`, com o visual do projeto (capa com o Guardião, cores e fontes do site). Depois de editar o manual, gere o PDF de novo:

```
python -m pip install markdown          # só na primeira vez
python docs/gerar_manual_pdf.py
```

O script usa o Microsoft Edge (ou o Google Chrome) instalado no computador para imprimir o PDF, e precisa de internet para carregar as fontes Cinzel. Ele não faz parte do app: o Vercel não o executa.

Apresentação do projeto (slides) e roteiro

`docs/Apresentação - O Guardião dos Dados.pdf` tem 14 slides (16:9, cerca de 10 minutos) com a identidade visual do jogo: contexto, o que é segurança da informação, dados de golpes no Brasil, ODS 16, ODS 10, golpes mais comuns e como se prevenir, o projeto, a validação, as referências e, no final, o link do app com QR Code. `docs/Roteiro da apresentação.pdf` traz o texto de fala de cada slide e o tempo sugerido.

Tudo vem de `docs/gerar_apresentacao_pdf.py`: os textos, números e fontes ficam na lista `SLIDES`, e o endereço do app em `LINK_APP`. Para mudar algo, edite o script e gere de novo:

```
python -m pip install segno            # só na primeira vez (QR Code)
python docs/gerar_apresentacao_pdf.py
```

Os dados dos gráficos são do Fórum Brasileiro de Segurança Pública (Anuários 2025 e 2026), do Cetic.br (TIC Domicílios 2024) e da Febraban (Pesquisa de Tecnologia Bancária 2025); as referências completas estão no slide 13. Se sair uma edição nova dessas pesquisas, atualize os números e as fontes juntos.

13. Cuidados e armadilhas

- **CSS não atualiza no navegador?** Depois de editar o `style.css`, aumente o número em **base.html** (ex.: `v='12'` → `v='13'`). Isso força navegadores e o Vercel a baixarem a versão nova.
- **Ponto e vírgula no `ESQUEMA`**: o SQL de criação é dividido em comandos pelo `;`. Não use `;` dentro dos comentários `--` do `ESQUEMA`, ou a criação do banco quebra.
- **Consultas SQL**: escreva sempre com `?` (ex.: `WHERE id = ?`) e passe os valores separados — a classe `Conexao` converte para o PostgreSQL. Nunca monte SQL juntando texto com valores digitados pelo usuário.
- `app = criar_app()` em `app.py` precisa continuar existindo com esse nome, ou o Vercel não encontra o aplicativo (`FLASK_ENTRYPOINT_NOT_FOUND`).
- **Dependências novas** precisam entrar no `requirements.txt`, senão funcionam no seu computador mas falham no Vercel.
- **Sessão no Vercel**: sem `QUIZ_SECRET_KEY` definida, jogadores podem perder a jornada no meio.
- **Dados "reiniciando" no Vercel?** É o sintoma de site sem PostgreSQL: sem `DATABASE_URL`, cada servidor do Vercel cria o próprio SQLite temporário em `/tmp`, e as jornadas somem a cada reinício ou troca de servidor. A área administrativa mostra um alerta vermelho nesse caso. Solução: conectar o banco Neon ao projeto (*Storage*) e fazer *Redeploy*.
- **Modo debug**: `python app.py` roda com `debug=True`, que recarrega o servidor ao salvar arquivos. É só para uso local; o Vercel não usa esse trecho.
- **Faixas de resultado em dois lugares**: as patentes (resultado.html) e as mensagens (jogo.py) usam as mesmas faixas de percentual (100 / 70 / 40). Se mudar uma, mude a outra.
- **Confirmações**: para pedir confirmação antes de enviar um formulário, use `data-confirmar="Texto da pergunta"` no `<form>` (não use `onsubmit="return confirm(...)"`, que mostra a janela padrão do navegador).
- **Aviso de privacidade**: ele descreve exatamente o que o app grava. Ao coletar um dado novo, atualize o texto em `menuprincipal.html` — é o princípio da transparência da LGPD.
- **Medidas dinâmicas no HTML**: não escreva `style="width: {{ valor }}%"` nos templates — o verificador de CSS do VS Code acusa erro (falso) e qualquer texto poderia entrar no CSS. Use `{{ estilo_percentual('width', valor) }}`.